

Credit Card Fraud Detection

Project Team

Furqan Tariq 21I-0857
Ali Mansoor 21I-0593
Fouad Ikram 21I-0682

Session 2021-2025

Supervised by

Dr. Arshad Islam

Co-Supervised by

-



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Module 1	3
1.3.2	Module 2	3
1.4	User Classes and Characteristics	3
2	Project Requirements	5
2.1	Use Cases	5
2.1.1	UC1: Detect Real-Time Fraudulent Transactions	5
2.1.2	UC2: Fraud Detection for Manually Uploaded Transactions	6
2.1.3	UC3: Generate Fraud Detection Report	7
2.1.4	UC4: View Fraud Detection Dashboard	8
2.1.5	UC5: Train GNN Model on User's Uploaded Dataset	9
2.2	Use Case Diagram	10
2.3	Functional Requirements	11
2.3.1	Module 1: Synthetic Data Generation and Preprocessing	11
2.3.2	Module 2: Graph Generation and GNN Model Training	11
2.4	Non-Functional Requirements	12
2.4.1	Reliability	12
2.4.2	Usability	12
2.4.3	Performance	13
2.4.4	Security	13
2.5	Domain Model	14
3	System Overview	15
3.1	Architectural Design	16
3.2	Design Models	16
3.3	Data Design	24
3.3.1	JSON Schema	24
	References	27

List of Figures

2.1	Use Case Diagram	10
2.2	Domain Model Diagram	14
3.1	Architecture Diagram	16
3.2	Activity Diagram for UC1	17
3.3	Activity Diagram for UC2	18
3.4	Activity Diagram for UC5	19
3.5	Data Flow Diagram for UC1	20
3.6	Data Flow Diagram for UC2	20
3.7	Data Flow Diagram for UC5	20
3.8	SD for UC1	21
3.9	SD for UC2	21
3.10	SD for UC3	22
3.11	SD for UC4	22
3.12	SD for UC5	23

List of Tables

Chapter 1

Introduction

The purpose of this proposal document is outlining the development process of a software system for credit card fraud detection. Credit card fraud is one of the most widespread crimes that poses a major threat to both customers and institutions worldwide. The total credit card losses due to fraud, globally amass up to billions of dollars. The proposed software system is based on utilizing the strengths of Graph Neural Network (GNNs) with a novel encoder decoder based architecture. The conventional Machine Learning (ML) algorithms that are currently being used, often rely on outdated techniques that are not quite efficient at capturing the complex and ever evolving nature of fraudulent transactions. Our system will use advanced GNN techniques to map connections between customers and merchants accurately which will result in bettering the accuracy of fraud detection in our system. The approach we will be using is taken from a research recently conducted at King Saud University in Saudi Arabia in 2024. The research tells the potential of GNNs in financial fraud detection because of their ability to capture both local and global information inside a graph data structure. [?].

1.1 Problem Statement

The credit card fraud detection system aims to overcome some of the limitations in fraud detection technologies. The majority of the available solutions yield very low recall rates, meaning they miss many true positive fraudulent transactions. They often use anonymized datasets that are far from real-world scenarios, and rely on traditional machine learning models that do not take into account the complex relationships between subjects involved in transactions. The software we will design will overcome these problems by solving the challenge of recognizing fraudulent transactions more efficiently using a GNN-based approach. This novel approach will help the system in learning complex patterns that are present within the transaction data, leading to higher fraud detection rates. Moreover,

our solution will also be built for an easy integration into a client's current transaction system to provide an user-friendly solution which will also be scalable for fraud detection in real-time.

1.2 Scope

This project aims to make an advanced software system for credit card fraud detection using an encoder-decoder Graph Neural Networks. Firstly, a synthetic dataset will be generated that has been designed to be as close as possible to real-life transaction scenarios, allowing robust model training. Furthermore, the project will extract and include crucial features through feature engineering, this will enhance the model's capability of differentiating between fair and fraudulent transactions. A key aspect of the project is to make a graph-based model of credit card transactions, where customers and merchants are represented as nodes, and transactions are represented as edges, enabling the system to inspect complex relationships within the data. The GNN-based fraud detector at the heart of the system will encode transaction data into compact representations and then decode this representation to identify fraudulent activities. To make training faster and improve accuracy, the project will implement batch normalization techniques, which will normalize activation within the GNN layers. The user interface will be designed to be easily used by clients. The user interface will provide visualization of transaction data, some performance metric dashboards, and also tools for investigating any fraudulent activities the system detects. The system will also be scalable which means it will be handling large chunks of data, and the system will also be able to adapt to evolving fraud tactics in real-time. Lastly, the system will have the ability to integrate with existing banking systems, which will facilitate easy acceptance by financial institutions. To conclude, this project's goal is finding a practical and efficient solution to the growing problem of credit card fraud by emphasizing on accuracy, scalability, and ease of use.

1.3 Modules

The project is divided into modules, each of which tackles a different component of the credit card fraud detection system. These modules will help break down the overall process of development into sections which ensures the essential elements of the system are developed, tested, and integrated. The team can concentrate on individual tasks, such as data preparation, model development, and user interface, while having a clear path to reach the overall project objectives. All this will be achieved by dividing the project into these modules.

1.3.1 Module 1

This module focuses on the steps required to build a reliable dataset, which are essential for training the fraud detection model. The process involves generating synthetic data that simulates real-world transactions, followed by meticulous data cleaning and pre-processing to make sure that the dataset is ready for model training.

1. Synthetic Data Generation
2. Feature Engineering
3. Data Cleaning
4. Data Pre-processing

1.3.2 Module 2

The primary task of this module is to construct a graph-based representation of the transaction data and then train the GNN model. The process of generating the graph involves making a model where customers and merchants are represented by nodes, and edges represent transactions. The GNN model is then trained on this graph data to point to patterns which are indicators of fraudulent activities.

1. Graph Generation
2. GNN Model Training and Evaluation

1.4 User Classes and Characteristics

User class	Description
Financial Institution Employees	Within financial firms and especially fraud teams are assigned to oversee and manage flagged transactions. About 200 users at different banks will engage with the system routinely. Each user will consistently inspect troubling transactions and respond effectively by initiating blocking cards or alerting customers approximately five to ten times per day. Users will engage with the system via desktop applications accessed within safe financial networks and require basic instruction to understand user interface features and insights offered by the GNN model.
System Administrators	System administrators operate as IT staff to manage and control the functioning of the fraud detection system. Ten to fifteen administrators will take charge of supervising backend systems as well as databases and integrations. They review the system consistently and deal with problems as well as update and backup data while ensuring it expands. These users will engage with safe backend interfaces and dashboards and need extensive technical training related to system design GNN models and infrastructure management.
Fraud Detection Software Developers	Developers of fraud detection software will modify or adapt the fraud detection system to fit the unique requirements of financial institutions. About 50 developers worldwide could utilize the system to include new functions and modify the GNN model. Frequent access to the codebase and APIs is needed for these users. To execute their tasks effectively they must possess extensive knowledge of GNN architecture along with the languages and frameworks such as PyTorch.
Executives and Regulatory Auditors	Senior team members would conduct regular tests of the system to ensure the system complies with legal norms. 10 to 20 senior users will view performance dashboards and reports once or twice monthly by engaging with secure reporting platforms. Familiarity with important metrics such as precision and recall will dominate the needs of these users with little training needed.

Chapter 2

Project Requirements

2.1 Use Cases

2.1.1 UC1: Detect Real-Time Fraudulent Transactions

Actor:

Financial Institution Employee

Description:

The system analyzes incoming transactions in real-time and flags suspicious transactions as potentially fraudulent.

Preconditions:

- The system must be actively receiving real-time synthetic transaction data.
- The fraud detection model must be trained and deployed.

Postconditions:

- Fraudulent transactions are flagged in real-time, and an alert is sent to the user.
- A fraud detection report is generated for later review.

Main Flow:

1. The system receives real-time transaction data from financial institutions.
2. The system processes each transaction and applies the fraud detection model.
3. The system flags transactions that are potentially fraudulent.

4. The system sends an alert to the user's dashboard indicating that a transaction is flagged as suspicious.
5. The user reviews the flagged transactions on the fraud detection dashboard.

Alternative Flows:

- **AF1.1:** If the fraud detection system is down, the system logs the failure, retries after a predefined interval, and alerts the user.

Extensions:

- **E1:** If the transaction is falsely flagged as fraudulent, the user can mark it as legitimate, and the system logs this information for future model tuning.

2.1.2 UC2: Fraud Detection for Manually Uploaded Transactions

Actor:

Financial Institution Employee

Description:

The user uploads a dataset of past transactions, and the system processes the data, identifying fraudulent and non-fraudulent transactions.

Preconditions:

- The user must have access to a dataset of past transactions in the required format.
- The fraud detection model must be trained and ready for batch processing.

Postconditions:

- Fraudulent transactions within the dataset are flagged.
- The user receives a downloadable fraud detection report for the uploaded dataset.

Main Flow:

1. The user navigates to the "Upload Transactions" section of the system.
2. The user uploads the transaction dataset file.
3. The system validates the dataset format and content.

4. The system processes the dataset using the fraud detection model.
5. The system flags fraudulent transactions and generates a fraud detection report.
6. The user downloads the report for further investigation.

Alternative Flows:

- **AF2.1:** If the dataset is in the wrong format or contains corrupt data, the system prompts the user to re-upload a valid file.
- **AF2.2:** If the system encounters a processing failure, it logs the error, retries processing, and notifies the user of the issue.

Extensions:

- **E2:** The user may choose to mark flagged transactions as legitimate, in which case the system logs this feedback for future tuning.

2.1.3 UC3: Generate Fraud Detection Report

Actor:

Financial Institution Employee

Description:

The user generates a detailed fraud detection report, including flagged transactions and performance metrics of the fraud detection system.

Preconditions:

- The system must have processed transaction data and fraud detection results available.

Postconditions:

- A fraud detection report is generated and available for download.

Main Flow:

1. The user navigates to the "Reports" section of the system.
2. The user selects the type of report (real-time or batch processing) and the time range.

3. The system generates the fraud detection report based on the selected criteria.
4. The system displays the generated report, and the user can view it on-screen or download it as a file.

Alternative Flows:

- **AF3.1:** If there is no data available for the selected time range, the system informs the user and prompts for a new range.
- **AF3.2:** If the report generation process fails, the system logs the error and notifies the user.

Extensions:

- **E3:** The user may request a custom report, including specific filters such as transaction type, amount, or geographical location.

2.1.4 UC4: View Fraud Detection Dashboard

Actor:

Financial Institution Employee

Description:

The user accesses the fraud detection dashboard to view real-time fraud detection results, system performance metrics, and transaction logs.

Preconditions:

- The system must have active fraud detection data available.

Postconditions:

- The user can view real-time fraud detection metrics and flagged transactions.

Main Flow:

1. The user navigates to the "Fraud Detection Dashboard" section.
2. The system displays the real-time fraud detection metrics, including flagged transactions, performance metrics, and transaction logs.
3. The user views the flagged transactions and investigates any alerts generated by the system.

4. The user can filter or search the dashboard to find specific transactions or metrics.

Alternative Flows:

- **AF4.1:** If the dashboard fails to load, the system logs the error, retries the loading process, and notifies the user.

Extensions:

- **E4:** The user may configure custom alerts and dashboard views to tailor the information displayed.

2.1.5 UC5: Train GNN Model on User's Uploaded Dataset

Actor:

Financial Institution Employee

Description:

The user uploads a dataset, and the system processes it to retrain the GNN model for fraud detection. The newly trained model is then used to detect fraud in future transactions.

Preconditions:

- The user must have access to a dataset in the required format (containing transactions, customer information, merchant data, and labels).
- The GNN model must be available for training.

Postconditions:

- The GNN model is successfully retrained using the uploaded dataset.
- The newly trained model is deployed for future fraud detection tasks.

Main Flow:

1. The user navigates to the "Train Custom GNN Model" section of the system.
2. The user uploads the dataset for GNN training.
3. The system validates the dataset format and checks for data integrity.
4. The system begins training the GNN model on the uploaded dataset.

5. Upon successful completion of the training, the system notifies the user.
6. The newly trained model is deployed, and the system starts using it for fraud detection.

Alternative Flows:

- **AF5.1:** If the dataset is not in the correct format or contains corrupt data, the system notifies the user and prompts them to re-upload a valid file.
- **AF5.2:** If the model training process fails due to system errors, the system logs the failure, retries the training process, and notifies the user.

Extensions:

- **E5:** The user may monitor the model training progress, viewing real-time updates on metrics like loss, accuracy, and time remaining for training completion.

2.2 Use Case Diagram

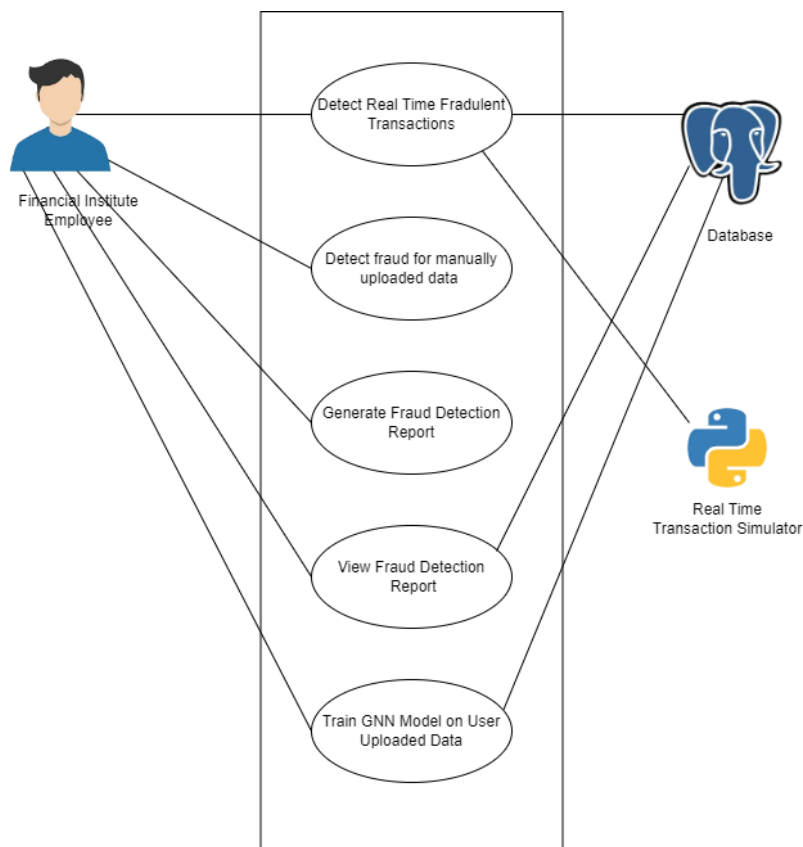


Figure 2.1: Use Case Diagram

2.3 Functional Requirements

2.3.1 Module 1: Synthetic Data Generation and Preprocessing

Following are the requirements for module 1 mapped against the features of the said module:

Feature 1: Synthetic Data Generation

1. The system shall generate synthetic transaction data, including transaction amount, time, customer ID, merchant ID, and fraud labels.
2. The system shall support generation of data at different scales, ensuring it mimics real-world transaction volumes.
3. The system shall generate fraudulent and non-fraudulent transactions in proportions that reflect real-world data.

Feature 2: Data Preprocessing

1. The system shall clean and preprocess the generated synthetic data, removing invalid or inconsistent entries.
2. The system shall normalize the transaction data to ensure consistency across all features.
3. The system shall split the dataset into training, validation, and test sets to prepare it for model training.

2.3.2 Module 2: Graph Generation and GNN Model Training

Following are the requirements for module 2 mapped against the features of the said module:

Feature 1: Graph Generation

1. The system shall convert transaction data into a graph structure, where nodes represent customers and merchants, and edges represent transactions.
2. The system shall ensure that the graph generation process can scale to handle large transaction datasets efficiently.
3. The system shall visualize the graph structure for easier understanding of relationships between customers and merchants.

Feature 2: GNN Model Training

1. The system shall implement an encoder-decoder Graph Neural Network (GNN) to train on the graph-structured data.
2. The system shall integrate batch normalization techniques to optimize GNN training and enhance model performance.
3. The system shall fine-tune the GNN's hyperparameters (e.g., learning rate, epochs) to maximize fraud detection accuracy.

2.4 Non-Functional Requirements

2.4.1 Reliability

- **NFR-R1:** The system shall have a mean time between failures (MTBF) of no less than 1,000 hours. A failure is defined as any event where the system is unable to detect, process, or respond to fraudulent transactions in real time.
- **NFR-R2:** In case of a system failure, the system shall automatically attempt to recover within 5 minutes by restarting the affected module (e.g., GNN model or graph generator) and logging the incident.
- **NFR-R3:** The system shall provide continuous monitoring and alert users in case of system failures, ensuring users are notified within 30 seconds of any failure.
- **NFR-R4:** System failures during real-time transaction processing shall lead to the preservation of transaction data and automatic resumption from the failure point once the system is back online.

2.4.2 Usability

- **NFR-U1:** The system shall allow a user to flag and investigate a fraudulent transaction within 5 clicks or interactions.
- **NFR-U2:** The user interface (UI) shall be designed for ease of use, such that a new user can complete fraud investigation tasks without training within 1 hour of initial interaction.
- **NFR-U3:** The system shall provide near real-time visual feedback (e.g., loading indicators, transaction status) to ensure users are aware of ongoing processes during fraud detection.

- **NFR-U4:** The system shall ensure that error messages are clear and actionable, providing users with steps to resolve issues within 10 seconds of error occurrence.

2.4.3 Performance

- **NFR-P1:** The system shall process up to 1,000 transactions per second (TPS) in real-time without performance degradation.
- **NFR-P2:** The GNN model shall detect fraudulent transactions with a response time of no more than 1 minute per transaction, regardless of dataset size.
- **NFR-P3:** The system shall support a maximum latency of 10 seconds for real-time dashboard updates when processing transaction data.
- **NFR-P4:** The system shall scale automatically to handle an increase in transaction volume up to 5x normal load without manual intervention.

2.4.4 Security

- **NFR-S1:** The system shall ensure that all transaction data is encrypted in transit and at rest using 256-bit encryption.
- **NFR-S2:** The system shall require multi-factor authentication (MFA) for all users to access critical components, such as fraud detection results and transaction logs.
- **NFR-S3:** The system shall automatically log out inactive users after 10 minutes of inactivity to prevent unauthorized access.

2.5 Domain Model

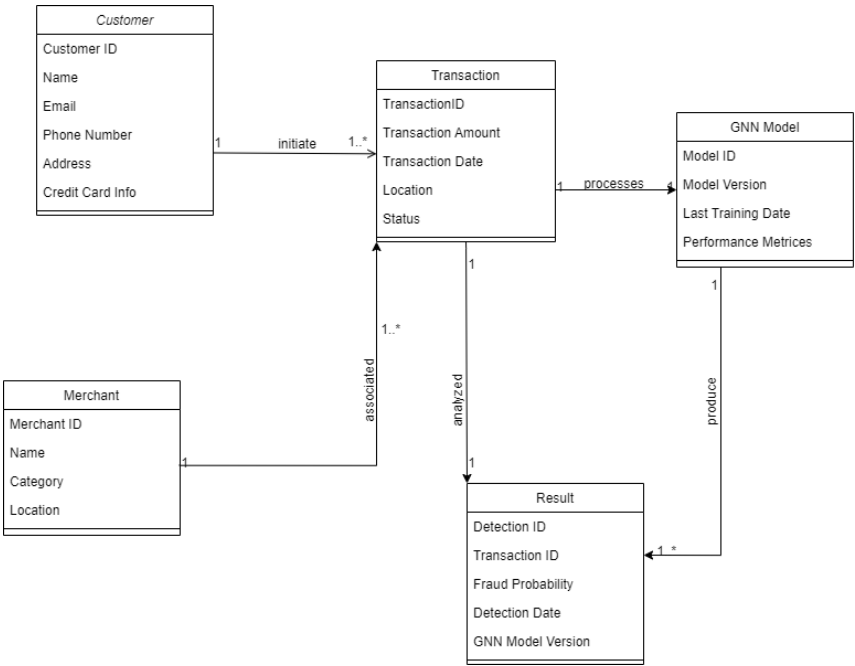


Figure 2.2: Domain Model Diagram

Chapter 3

System Overview

The user interface is a gateway through which a user communicates with the system; it gives a friendly approach to the system's functionalities. It makes simple jobs such as uploading datasets, often in CSV format, possible while guaranteeing a constant inflow of fresh data for analysis.

The application logic is at the center of the system and controls data processing as well as key conversions. This involves data pre-processing such as parsing, cleaning and feature extraction in order to prepare the data for fraud detection. Using technologies like GNNs the logic looks for fraud and finds different patterns and outliers in the transactional data.

The data storage component of the system is the primary to keep, safeguard, and retrieve data. It retains raw transaction data for perpetuity to be used for analysis later and also saves the output of fraud detection, such as suspicious transactions and fraud patterns. They are useful records, which could be further analyzed and which help to make informed decisions.

3.1 Architectural Design

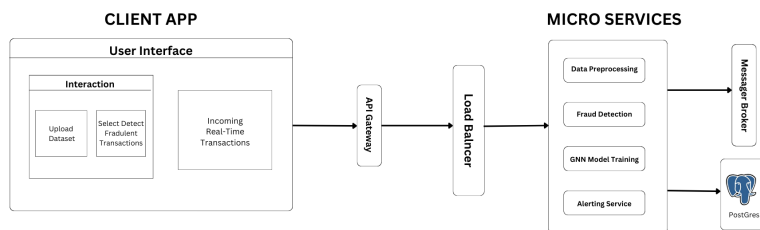


Figure 3.1: Architecture Diagram

3.2 Design Models

We used a procedural approach so our design models include, Activity Diagram, Data Flow Diagram and System Level Sequence Diagram

- **Activity Diagram**

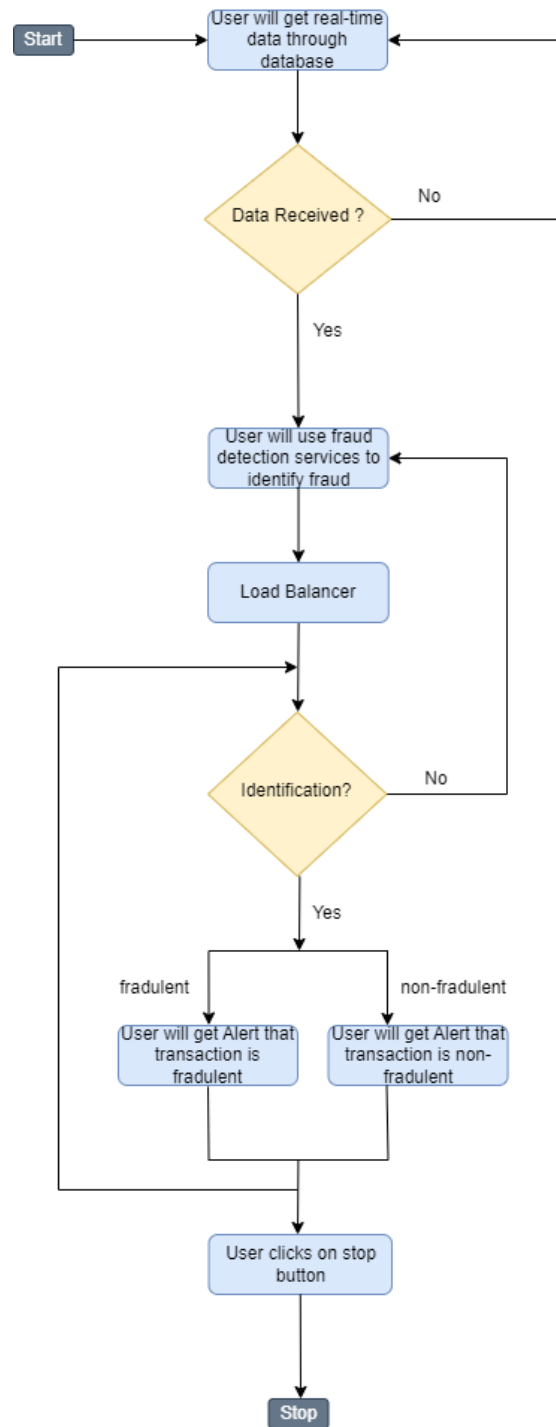


Figure 3.2: Activity Diagram for UC1

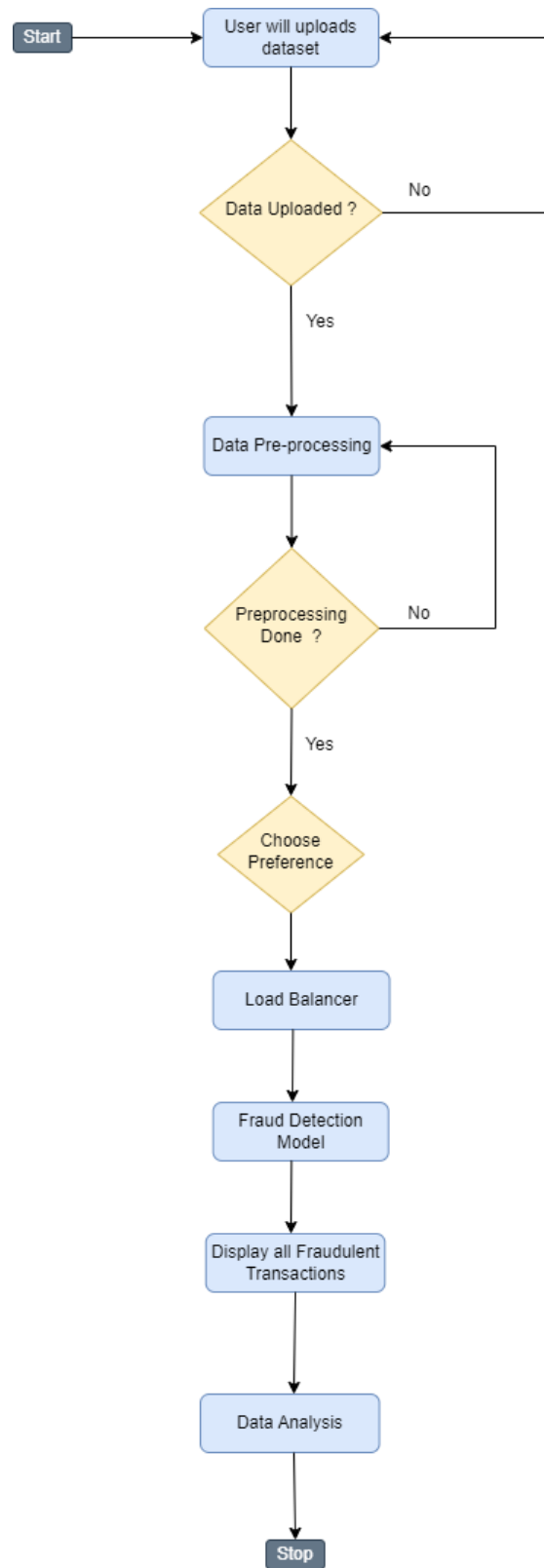


Figure 3.3: Activity Diagram for UC2

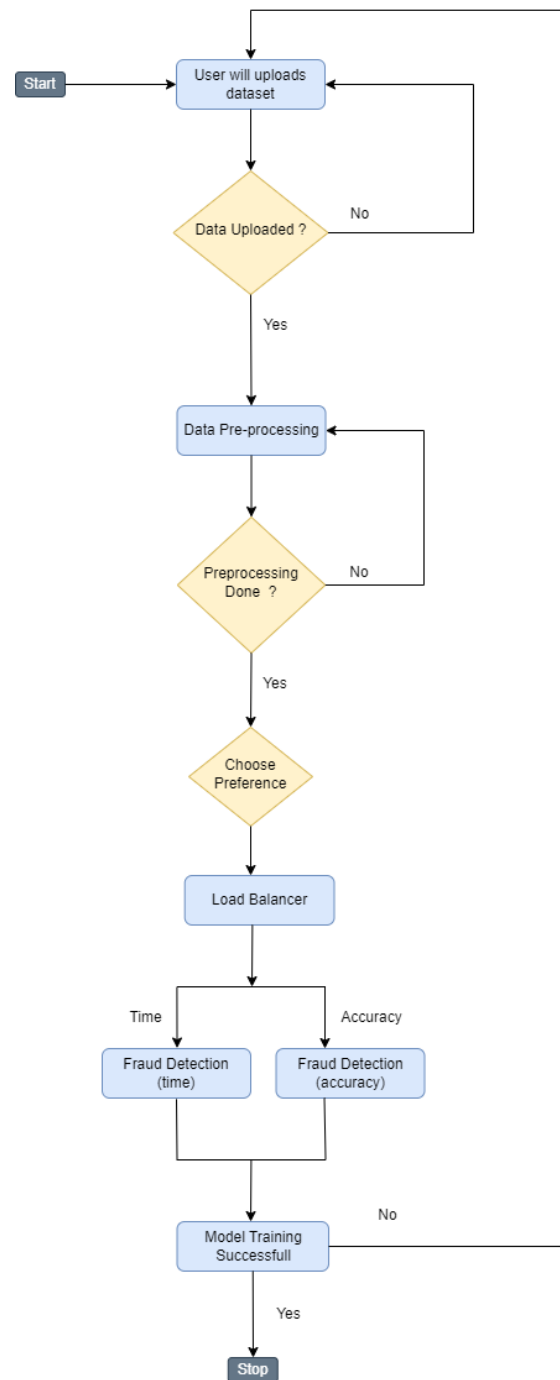


Figure 3.4: Activity Diagram for UC5

• Data Flow Diagram

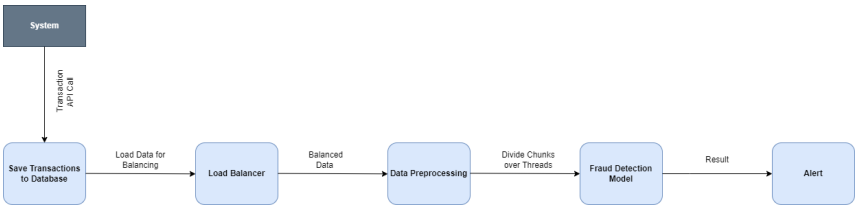


Figure 3.5: Data Flow Diagram for UC1

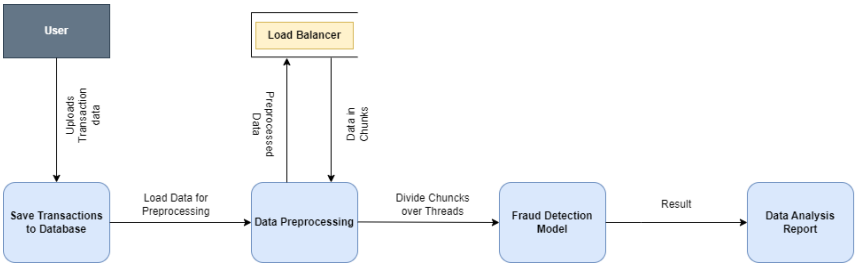


Figure 3.6: Data Flow Diagram for UC2

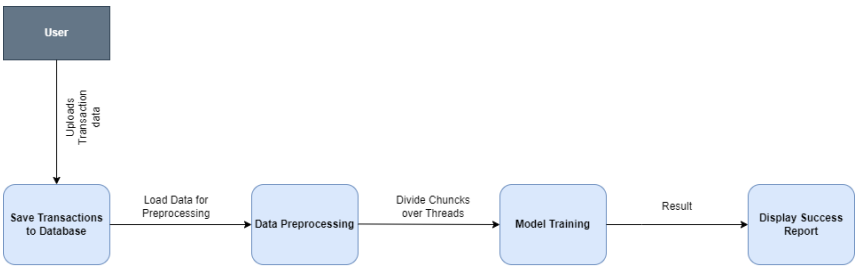


Figure 3.7: Data Flow Diagram for UC5

• System-level Sequence Diagram

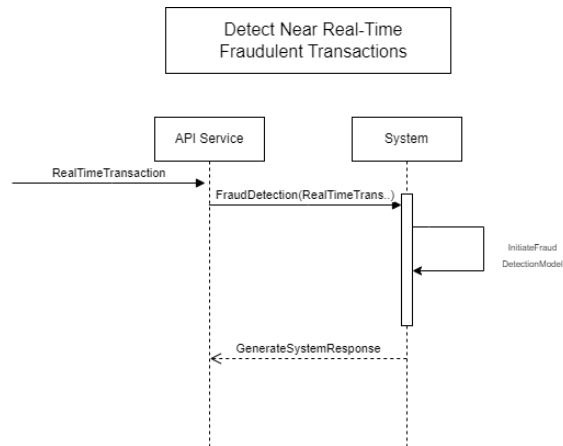


Figure 3.8: SD for UC1

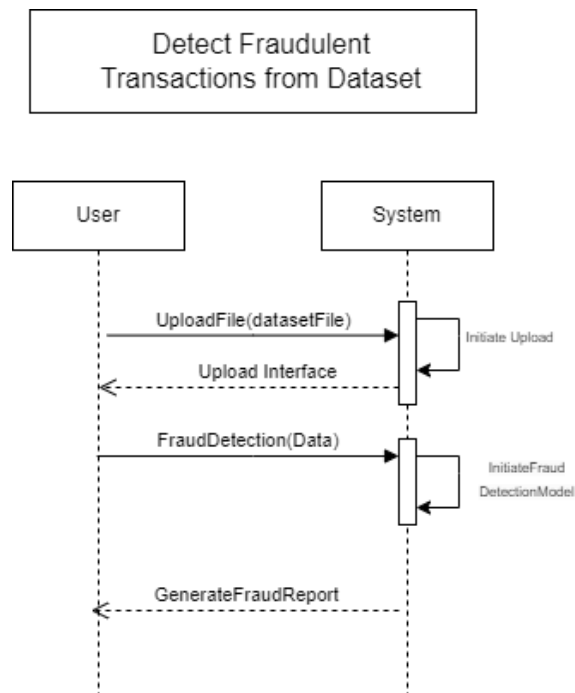


Figure 3.9: SD for UC2

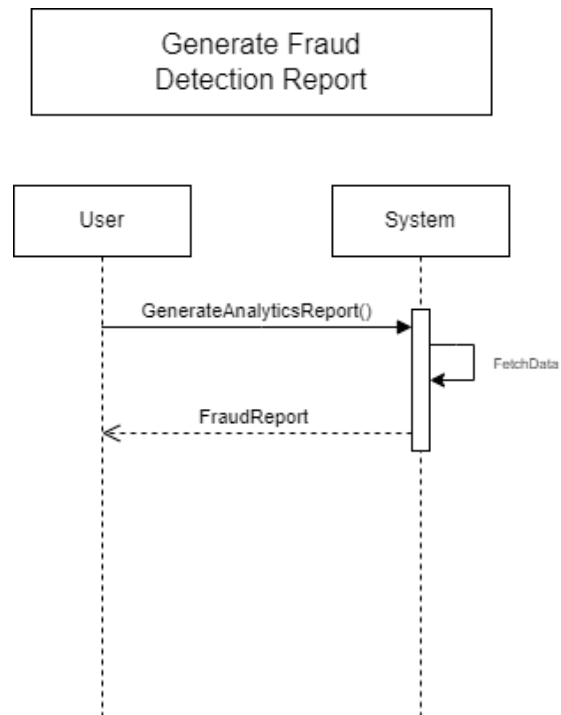


Figure 3.10: SD for UC3

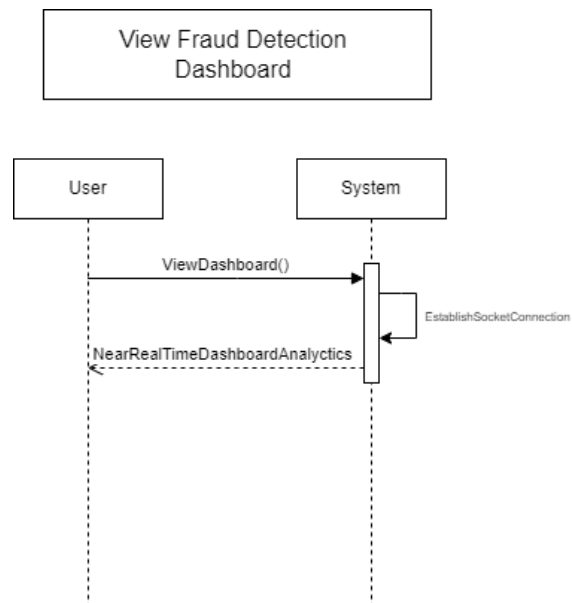


Figure 3.11: SD for UC4

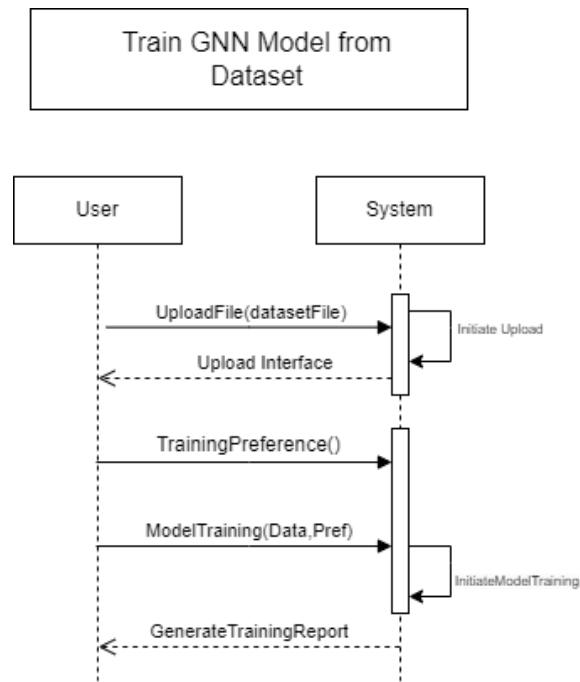


Figure 3.12: SD for UC5

3.3 Data Design

Our FYP needs a database, but it does not need a full infrastructure because only the transaction data will be stored. The focus of our project is on two main sources: a large initial data set in CSV format and real-time streaming transactions for financial fraud identification. Since we are not going to deal with many data relationships and do not require large storage, there is no need to create an Entity-Relationship Diagram (ERD) for this context. However, we will provide the JSON schema of the structure of our dataset for the sake of clear representation and referencing.

3.3.1 JSON Schema

```
{
  "type": "object",
  "properties": {
    "ssn": {
      "type": "string",
      "description": "The social security number of the credit card holder"
    },
    "cc_num": {
      "type": "string",
      "description": "The credit card number"
    },
    "first": {
      "type": "string",
      "description": "The first name of the credit card holder"
    },
    "last": {
      "type": "string",
      "description": "The last name of the credit card holder"
    },
    "gender": {
      "type": "string",
      "enum": ["male", "female", "other"],
      "description": "The gender of the credit card holder"
    },
    "street": {
      "type": "string",
      "description": "The street address of the credit card holder"
    },
    "city": {
      "type": "string",
      "description": "The city of the credit card holder"
    },
    "state": {
```

```

    "type": "string",
    "minLength": 2,
    "maxLength": 2,
    "description": "The state abbreviation of the credit card holder's address"
  },
  "zip": {
    "type": "string",
    "pattern": "^[0-9]{5}(?:-[0-9]{4})?$",
    "description": "The ZIP code of the credit card holder's address"
  },
  "lat": {
    "type": "number",
    "description": "The latitude coordinate of the credit card holder's address"
  },
  "long": {
    "type": "number",
    "description": "The longitude coordinate of the credit card holder's address"
  },
  "city_pop": {
    "type": "integer",
    "description": "The population of the city where the credit card holder resides"
  },
  "job": {
    "type": "string",
    "description": "The job title or occupation of the credit card holder"
  },
  "dob": {
    "type": "string",
    "format": "date",
    "description": "The date of birth of the credit card holder"
  },
  "acct_num": {
    "type": "string",
    "description": "The account number associated with the credit card"
  },
  "profile": {
    "type": "string",
    "description": "A profile identifier or description for the credit card holder"
  },
  "trans_num": {
    "type": "string",
    "description": "The unique transaction number"
  },
  "trans_date": {
    "type": "string",
    "format": "date",
    "description": "The date of the transaction"
  },

```

```
"trans_time": {
  "type": "string",
  "pattern": "^(0[0-9]|1[0-2]):[0-5][0-9] (AM|PM)$",
  "description": "The time of the transaction in 12-hour format"
},
"unix_time": {
  "type": "integer",
  "description": "The Unix timestamp of the transaction"
},
"category": {
  "type": "string",
  "description": "The category of the transaction (e.g., grocery, gas)"
},
"amt": {
  "type": "number",
  "description": "The amount of the transaction"
},
"is_fraud": {
  "type": "boolean",
  "description": "Indicates whether the transaction is fraudulent"
},
"merchant": {
  "type": "string",
  "description": "The name of the merchant where the transaction took place"
},
"merch_lat": {
  "type": "number",
  "description": "The latitude coordinate of the merchant's location"
},
"merch_long": {
  "type": "number",
  "description": "The longitude coordinate of the merchant's location"
}
},
"required": [
  "ssn", "cc_num", "first", "last", "gender", "street", "city", "state",
  "zip", "lat", "long", "city_pop", "job", "dob", "acct_num", "profile",
  "trans_num", "trans_date", "trans_time", "unix_time", "category", "amt",
  "is_fraud", "merchant", "merch_lat", "merch_long"
]
}
```

Bibliography